Implementation and Verification of a CAN-FD Bus Transceiver in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

1 Design and Architecture

1.1 System Overview

A complete CAN node decomposes into a TX path and an RX path, coordinated by shared Fault Confinement Entity (FCE) and Physical Medium Attachment (PMA) control, as shown in Figure 1. Each path spans three sub-layers - LLC (Section 1.5), MAC (Section 1.6), and PCS (Section 1.7) - with the LLC frame format defined in Section 1.2 and interface bundles defined in Section 1.3. A protocol-driven type system (Section 1.4) underpins all inter-module interfaces, encoding ISO semantics directly into the VHDL type hierarchy.

1.2 LLC Frame Format

The `LLC Frame` format used by the existing CAN-bus implementation (`can_bus_controller`) is depicted in Figure 2 with the ID byte format depicted in Table 1. A revised `LLC Frame` supporting FD is depicted in Figure 3. The revised format adds a 3-bit `FMT` [1, Sec. 6.4.3] field to the DLC byte (`LLC Frame` byte 4),

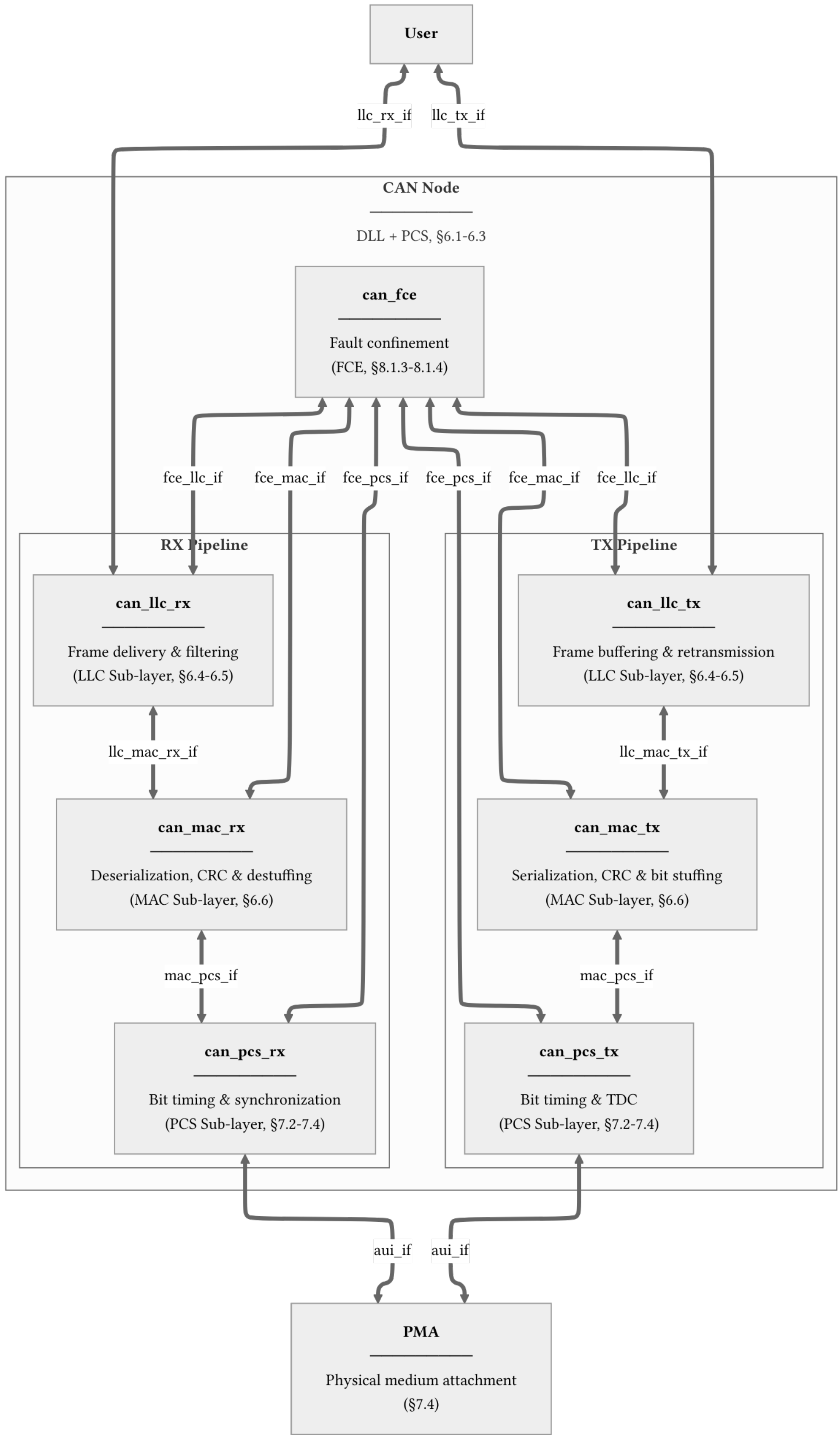

Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for llc_tx_if (Table 2), llc_rx_if (Table 3), llc_mac_tx_if (Table 4), llc_mac_rx_if (Table 5), mac_pcs_if (Table 6), aui_if (Table 7), fce_llc_if (Table 8), fce_mac_if (Table 9), and fce_pcs_if (Table 10).

expands the data field to 64 bytes, and repurposes reserved bits in the last byte for BRS and ESI flags.

The `FMT` encodes the supported frame formats as ‘`000`’ = CB, ‘`100`’ = CE, ‘`010`’ = FB, ‘`110`’ = FE. Accordingly, for the frame content to be self-consistent the IDE bit must be set to `0` for `FMT` = CB/FB and `1` for `FMT` = CE/FE. The revised format is designed to be backward compatible. An implementation that only supports Classic frames can simply ignore the `FMT` bits and treat all frames as Classic, while an implementation that supports FD can use the `FMT` field and the additional control bits (BRS and ESI) to distinguish frame types without affecting the existing ID and data field structure.

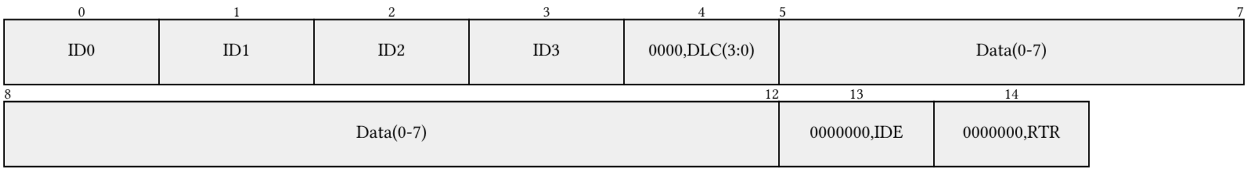

Figure 2: Current LLC frame format (15 bytes). ID byte encoding is defined in Table 1.

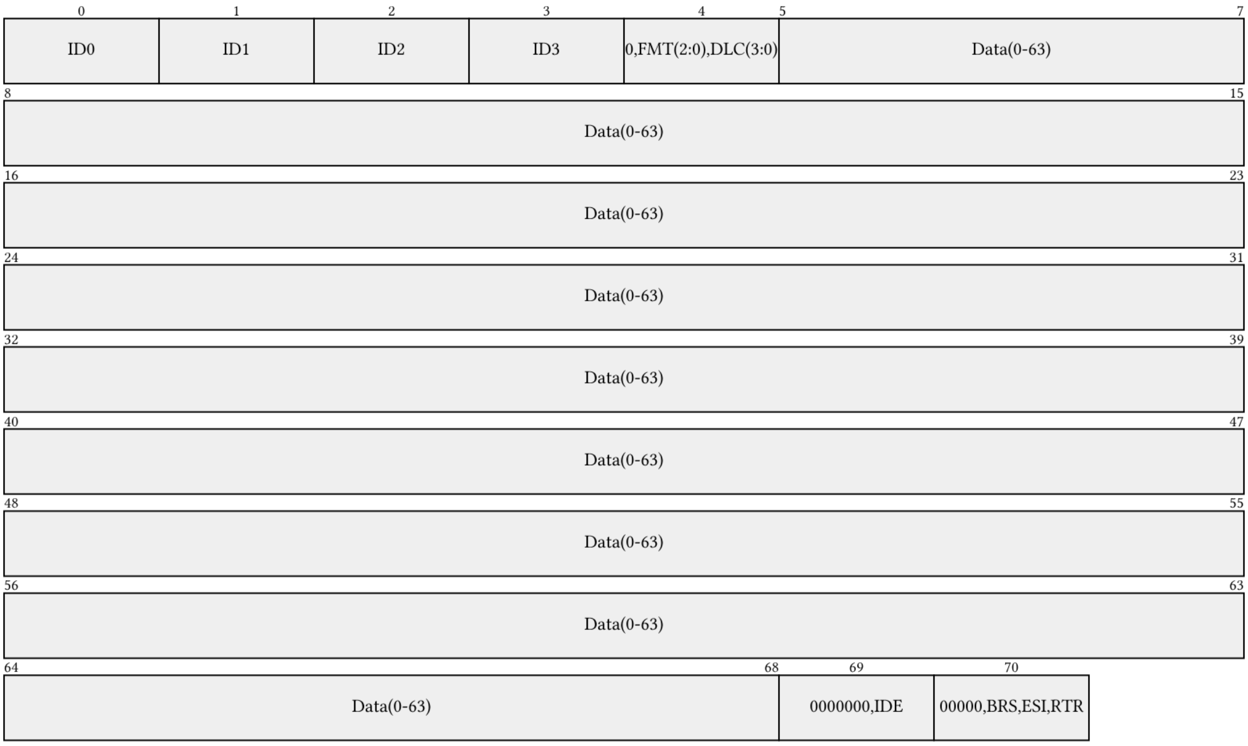

Figure 3: Revised LLC frame format with FD support, shown at maximum length (71 bytes). The FMT field selects frame type; BRS and ESI repurpose reserved bits in the final byte. ID byte encoding is defined in Table 1.

Table 1: ID byte encoding for the LLC frame formats shown in Figure 2 and Figure 3.

Format	Byte ID0	Byte ID1	Byte ID2	Byte ID3
Basic	‘`00000000`’	‘`00000000`’	‘`00000`’ & ‘`ID(10-8)`’	‘`ID(7-0)`’
Extended	‘`000`’ & ‘`ID(28:24)`’	‘`ID(23-16)`’	‘`ID(15-8)`’	‘`ID(7-0)`’

1.3 Interface Definition Tables

The interface bundles shown in Figure 1 are defined in full below, with each signal mapped to its corresponding ISO 11898-1 service primitive, [1]. This normative anchoring provides a direct traceability path from protocol clauses to VHDL ports. The types used in these interfaces are defined in Section 1.4.

The semantic protocol types described in Section 1.4 are used internally within the MAC layer and at the MAC-PCS boundary, where frame context must be carried alongside each bit. The LLC layer operates purely on bytes - the LLC frame format (Section 1.2) is defined in terms of plain data fields with no protocol-semantic typing. The AUI interface (`aui_if`) uses `std_logic` throughout, as it crosses outside the ISO protocol domain into the physical medium. The one exception is `transfer_status_t`, which propagates a frame outcome from the MAC layer back through the LLC to the user. The LLC-User interface (`llc_tx_if`, `llc_rx_if`) is implemented as an Avalon-ST byte stream to maintain backward compatibility with the existing CAN bus controller, which uses the same `valid`/`ready`/`startofpacket` handshake convention.

Table 2: Interface definition for `llc_tx_if`. Implements `L_Data.Request` as an Avalon-ST byte stream. `L_Data.AbortRequest` is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`), `llc_tx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` asserted on first byte
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`)	`valid` high; `sop` and `eop` deasserted on intermediate bytes
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`), `llc_tx_if.eop` (`std_logic`)	`valid` high with byte on data; `eop` asserted on last byte
6.4.5.5.2	`L_Data.Request`		Flow control	`LLC -> User`	`llc_tx_if.ready` (`std_logic`)	Asserted by LLC when able to consume next byte
6.4.5.5.3	`L_Data.AbortRequest`	`Handle`	Abort pending frame transfer	`User -> LLC`	`llc_tx_if.abort_request` (`std_logic`)	Pulse when user wants to cancel an in-progress transfer

Table 3: Interface definition for `llc_rx_if`. Implements `L_Data.Indication` as an Avalon-ST byte stream. `Timestamp` is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`), `llc_rx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` asserted on first byte

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`)	`valid` high; `sop` and `eop` deasserted on intermediate bytes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`), `llc_rx_if.eop` (`std_logic`)	`valid` high with byte on data; `eop` asserted on last byte
6.4.5.5.5	`L_Data.Indication`		Flow control	`User -> LLC`	`llc_rx_if.ready` (`std_logic`)	Asserted by user when able to consume next byte
6.4.5.5.4	`L_Data.Confirm`	`Transfer_Status`, `Timestamp`, `Handle`	Report outcome of prior `L_Data.Request`	`LLC -> User`	`llc_rx_if.transfer_status` (`transfer_status_t`)	Issued on frame completion, loss, or error

Table 4: Interface definition for `llc_mac_tx_if`. Frame length is self-describing from the DLC config bytes; the MAC serializer does not consume `eop`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.3, 6.6.4.2	`DLL SDU`	`LLC Frame`	Transfer LLC frame to MAC for serialization	`LLC -> MAC`	`llc_mac_tx_if.data` (`byte_t`), `llc_mac_tx_if.valid` (`std_logic`), `llc_mac_tx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` marks first byte of new frame
6.3, 6.6.4.2	`DLL SDU`		Flow control	`MAC -> LLC`	`llc_mac_tx_if.ready` (`std_logic`)	Asserted by MAC serializer when able to consume next byte
6.6.4.2	Interface control information	`Transfer_Status`	Report frame transmission outcome	`MAC -> LLC`	`llc_mac_tx_if.transfer_status` (`transfer_status_t`)	Updated by MAC on completion, loss, or error

Table 5: Interface definition for `llc_mac_rx_if`. Carries the reconstructed LLC frame from `can_mac_rx` to `can_llc_rx` (Section 1.6.2). Full decomposition of RX sub-module interfaces is deferred to future work.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.6.4.3, 6.6.9	`DLL SDU`	Reconstructed `LLC Frame`	Transfer reconstructed LLC frame to LLC	`MAC -> LLC`		Issued when MAC has reconstructed a frame from the received bitstream
6.6.3	Time reference notification	`SOF/frame-valid indication`	Provide timing reference for timestamping	`MAC -> LLC`		Generated for transmitted/received DF/RF

Table 6: Interface definition for `mac_pcs_if`. The `D_Transmit` status is encoded directly in `mac_pcs_if.data.bit_name` rather than as a separate signal - the PCS derives data-phase entry and exit from the protocol-semantic bit name, eliminating the need for a redundant boolean flag.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.2	`PCS_Data.Request`	`Output_Unit`	Present next bit for transmission	`MAC -> PCS`	`mac_pcs_if.data` (`mac_frame_bit_t`)	MAC holds bit stable; PCS samples `data` at each bit boundary autonomously
7.2.1, 7.2.5	`PCS_Status.Transmitter`	`D_Transmit`	Indicate FD data-phase interval to PCS	`MAC -> PCS`	`mac_pcs_if.data.bit_name` (`mac_frame_bit_name_t`)	PCS enters data phase at `fdf_bit` SP, exits at `crc_delimiter_bit` or error flag boundary
7.2.1, 7.2.3	`PCS_Data.Indicate`	`Input_Unit`	Indicate bus polarity at sample point	`PCS -> MAC`	`mac_pcs_if.bus_polarity` (`polarity_t`), `mac_pcs_if.sample_strobe` (`std_logic`), `mac_pcs_if.strobe_type` (`strobe_type_t`), `mac_pcs_if.fifo_index` (`integer`)	`sample_strobe` pulses once per SP/SSP; `strobe_type` distinguishes SP from SSP for TDC

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.6	`PCS_Status.Receiver`	`D_Receive`	Indicate FD data-phase interval to PCS	`MAC -> PCS`	not yet implemented	Asserted during FD data-phase reception

Table 7: Interface definition for `aui_if`. All signals are `std_logic`, as this interface crosses from the ISO protocol domain into the physical medium.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.4.2.1	`output symbol`	`Dominant/recessive symbol`	Drive physical output symbol	`PCS -> PMA`	`aui_if.tx` (`std_logic`)	Updated on each `Output_Unit` request
7.4.2.2, 8.1.3.4	`bus_off symbol`	`Bus-off control`	Switch node off bus	`PCS -> PMA`	`aui_if.bus_off` (`std_logic`)	On `Bus_off_request` from FCE
7.4.2.3, 8.1.3.4	`bus_off_release symbol`	`Bus-off release control`	Release node from bus-off	`PCS -> PMA`	`aui_if.bus_off_release` (`std_logic`)	On `Bus_off_release_request` from FCE
7.4.3	`input symbol`	`Dominant/recessive symbol`	Indicate physical input symbol	`PMA -> PCS`	`aui_if.rx` (`std_logic`)	Continuously driven by PMA

Table 8: Interface definition for `fce_llc_if`. Carries bus-off status and mode-request handshake between the FCE and LLC layers [1, Sec. 8.1.3.2].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.2	`Normal_mode_request`	`Mode request`	Request reset to normal mode	`LLC -> FCE`	`fce_llc_if.normal_mode_request` (`boolean`)	Issued on startup/restart

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.2	`Normal_mode_response`	`Mode response`	Acknowledge normal-mode request	`FCE -> LLC`	`fce_llc_if.normal_mode_response` (boolean)	Returned after FCE processing
8.1.3.2	`Bus_off`	`Bus-off status`	Indicate node is bus-off	`FCE -> LLC`	`fce_llc_if.bus_off` (boolean)	Asserted on bus-off transition

Table 9: Interface definition for `fce_mac_if`. The MAC reports error events and frame outcomes to the FCE; the FCE returns the current error-active/passive state to the MAC [1, Sec. 8.1.3.3].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	`Transmit/receive`	`Transfer mode context`	Report current TX/RX context	`MAC -> FCE`	`fce_mac_if.transmitting` (boolean)	Updated with MAC transfer context
8.1.3.3	`Error`	`Error event`	Report detected protocol error	`MAC -> FCE`	`fce_mac_if.error` (boolean)	Pulse on bit/stuff/CRC/form/ACK error
8.1.3.3	`Primary_error`	`Primary error event`	Report primary error condition	`MAC -> FCE`	`fce_mac_if.primary_error` (boolean)	Pulse on primary error condition
8.1.3.3	`Error/overload flag`	`EF/OF state`	Report EF/OF transmission state	`MAC -> FCE`	`fce_mac_if.sending_error_flag` (boolean)	Asserted during EF/OF transmission
8.1.3.3	`Counters_unchanged`	`Counter-update qualifier`	Qualify counter exception path	`MAC -> FCE`	`fce_mac_if.counters_unchanged` (boolean)	Asserted on rule-c exception cases
8.1.3.3	`Error_delimiter_too_la`	`Late delimiter event`	Report late error-delimiter condition	`MAC -> FCE`	`fce_mac_if.error_delimiter_too_` (boolean)	Asserted on late delimiter condition
8.1.3.3	`Successful_transfer`	`Transfer completion event`	Report successful TX/RX completion	`MAC -> FCE`	`fce_mac_if.successful_transfer` (boolean)	Pulse on successful frame transfer

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	`Error_passive_response`	`State response`	Report entry into error-passive state	`MAC -> FCE`	`fce_mac_if.error_passive_respor` (`boolean`)	On state transition completion
8.1.3.3	`Error_active_response`	`State response`	Report return to error-active state	`MAC -> FCE`	`fce_mac_if.error_active_respons` (`boolean`)	On state transition completion
8.1.3.3	`Error_passive_request`	`State request`	Request MAC enter error-passive state	`FCE -> MAC`	`fce_mac_if.error_passive` (`boolean`)	On TEC/REC threshold crossing
8.1.3.3	`Error_active_request`	`State request`	Request MAC return to error-active state	`FCE -> MAC`	`fce_mac_if.error_active` (`boolean`)	On TEC/REC recovery

Table 10: Interface definition for `fce_pcs_if`. Carries bus-off and bus-off-release request/response handshakes between the FCE and PCS layers [1, Sec. 8.1.3.4].

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.4	`Bus_off_request`	`Bus-off request`	Request node switch-off from bus	`FCE -> PCS`	`fce_pcs_if.bus_off_request` (`boolean`)	On bus-off transition condition
8.1.3.4	`Bus_off_release_request`	`Bus-off release request`	Request node re-enable from bus-off	`FCE -> PCS`	`fce_pcs_if.bus_off_release_requ` (`boolean`)	On restart/reintegration
8.1.3.4	`Bus_off_response`	`Bus-off response`	Acknowledge bus-off request	`PCS -> FCE`	`fce_pcs_if.bus_off_response` (`boolean`)	Returned after bus-off action
8.1.3.4	`Bus_off_release_respons`	`Bus-off release response`	Acknowledge bus-off-release request	`PCS -> FCE`	`fce_pcs_if.bus_off_release_resp` (`boolean`)	Returned after release action

1.4 Protocol-Driven Type System

The type system encodes protocol semantics directly into the interface bundles defined in Section 1.3. This makes the protocol semantics visible in the implementation and in simulation waveforms. As shown in Figure 4, the hierarchy is rooted in ISO-derived protocol constants, from which frame layout constants, semantic enumeration types, and composite record types are derived.

1.5 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

1.5.1 `can_llc_tx`

`can_llc_tx` buffers an incoming LLC frame from the user, streams it byte-by-byte to the MAC serializer, and manages retransmission on bus disturbance up to the ISO-mandated limit [1, Sec. 6.4.5] and 6.5.

1.5.2 `can_llc_rx`

`can_llc_rx` receives a reconstructed LLC frame from the MAC layer, applies acceptance filtering, and delivers accepted frames to the user over the `llc_rx_if` Avalon-ST stream [1, Sec. 6.4.5].

1.6 MAC Sub-layer

The MAC sub-layer is the core of the protocol logic, responsible for bit serialization, CRC generation, bit stuffing, and frame-level error detection. It coordinates closely with the FCE (Section **??**) for error counter management and node-state transitions (Error Active/Passive/Bus Off), and with the PCS (Section 1.7) for sample-point-driven bit output.

The earlier CAN bus controller concentrated TX, RX, MAC, and FCE logic in a single monolithic FSM (`can_fsm`), with the sub-functions - serialization (`can_ast_to_serial`), bit stuffing (`can_stuff_bit_gen`), and CRC (`gen_crc`) - implemented in satellite modules driven directly by it. PCS timing logic was implemented in `can_node_clock`, which fed sample-point and transmit pulses into `can_fsm` - a strategy retained in the current design.

The current design restructures these modules around the ISO 11898-1 [1] layer boundaries. On the TX side the mapping is direct: `can_ast_to_serial` becomes `can_mac_ser_tx` (Section 1.6.1.3), `can_stuff_bit_gen` becomes `can_mac_bs_tx` (Section 1.6.1.4), `gen_crc` becomes `can_mac_crc_tx` (Section 1.6.1.5), and `can_node_clock` becomes `can_pcs_tx` (Section 1.7.1). The RX-side counterparts - deserialization (`can_serial_to_ast`), bit destuffing, and CRC checking - will map to submodules within `can_mac_rx` (Section 1.6.2). Fault confinement, previously embedded in `can_fsm`, is separated into its own layer.

1.6.1 `can_mac_tx`

The `can_mac_tx` (Figure 5) is composed of four main components:

1. **can_mac_ser_tx**: Converts LLC bytes into a serial bit stream with polarity information
2. **can_mac_fsm_tx**: Coordinates frame transmission, controls all submodules
3. **can_mac_bs_tx**: Implements CAN/CAN-FD bit stuffing rules
4. **can_mac_crc_tx**: Generates CRC-15/CRC-17/CRC-21 based on frame type

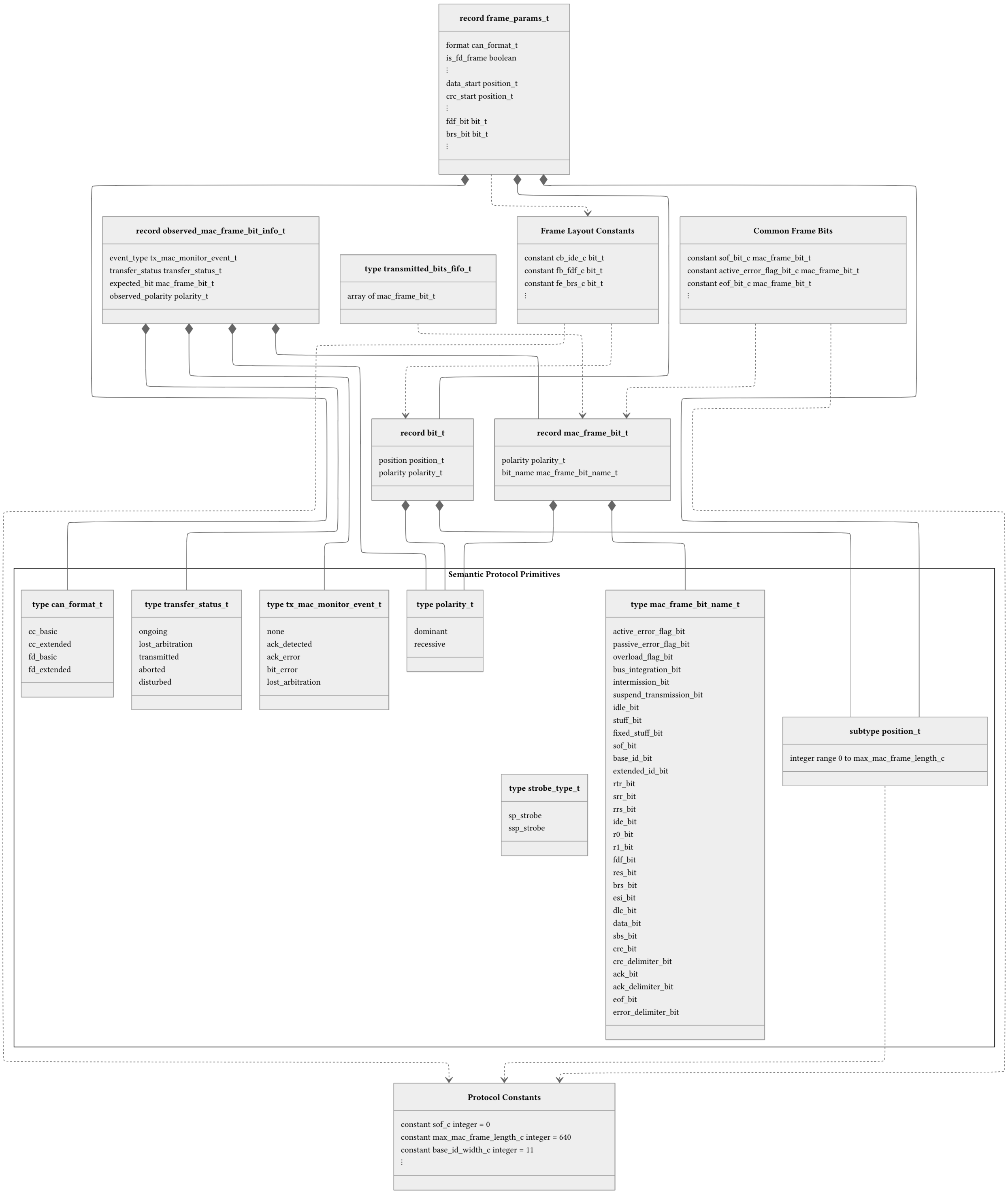

Figure 4: Type and constant hierarchy. The Semantic Protocol Primitives namespace groups the enumeration types and subtypes that comprise the protocol semantic primitives. Compound record types compose these primitives: bit_t pairs a bit position with a polarity and underpins the frame layout constants. mac_frame_bit_t carries semantic context by combining a polarity with a protocol bit name, so each transmitted bit remains identifiable throughout the design. frame_params_t aggregates format flags, field boundary positions, and format-specific control bit positions, computed once per frame (Section 1.6.1.3). observed_mac_frame_bit_info_t and transmitted_bits_fifo_t support bus monitoring and TDC-delayed bit comparison (Section 1.6.1.2). Constant groups derive from the root protocol constants.

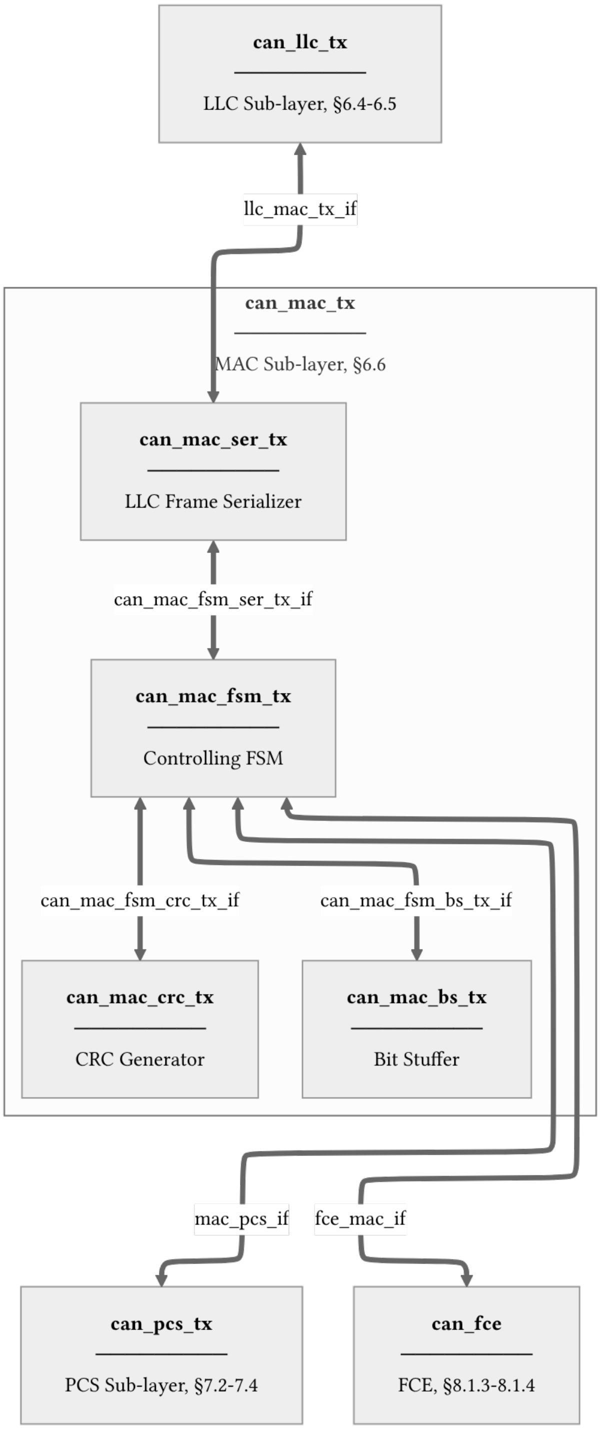

Figure 5: can_mac_tx architecture showing internal component interconnections and external interfaces. Internal interface definitions are provided for can_mac_fsm_ser_tx_if (Table 11), can_mac_fsm_bs_tx_if (Table 12), and can_mac_fsm_crc_tx_if (Table 13).

1.6.1.1 `can_mac_tx` Internal Interfaces The interfaces between MAC sub-components are implementation-defined and carry no direct ISO service primitive mapping. Table 11, Table 12, and Table 13 define each bidirectional bundle; the Direction column identifies the driving component for each field.

Table 11: interface definition for `can_mac_fsm_ser_tx_if`, connecting `can_mac_ser_tx` and `can_mac_fsm_tx` (see Figure 5).

field	type	direction	description
`data`	`polarity_t`	`ser` → `fsm`	current bit polarity; fsm reads this when `valid` is asserted
`valid`	`boolean`	`ser` → `fsm`	asserted while a bit is available for the fsm to consume
`frame_params`	`frame_params_t`	`ser` → `fsm`	frame parameters computed from the two config bytes; valid for the lifetime of the frame
`ready`	`boolean`	`fsm` → `ser`	pulsed when the fsm has consumed the current bit; advances the serializer to the next bit
`transfer_status`	`transfer_status_t`	`fsm` → `ser`	frame outcome; any non-`ongoing` value terminates serialization

Table 12: interface definition for `can_mac_fsm_bs_tx_if`, connecting `can_mac_fsm_tx` and `can_mac_bs_tx` (see Figure 5).

field	type	direction	description
`data`	`polarity_t`	`fsm` → `bs`	bit polarity fed into the bit stuffer
`valid`	`boolean`	`fsm` → `bs`	pulsed when a new bit is presented to the stuffer
`start`	`boolean`	`fsm` → `bs`	pulsed at frame start to reinitialize the stuffer state
`data`	`polarity_t`	`bs` → `fsm`	polarity of the required stuff bit
`valid`	`boolean`	`bs` → `fsm`	asserted when a stuff bit insertion is required
`sbc`	`std_logic_vector(3:0)`	`bs` → `fsm`	gray-coded stuff bit count with parity, used in the fd fixed-stuffing region

Table 13: interface definition for `can_mac_fsm_crc_tx_if`, connecting `can_mac_fsm_tx` and `can_mac_crc_tx` (see Figure 5).

field	type	direction	description
`crc_poly_select`	`std_logic_vector(1:0)`	`fsm → crc`	selects the active crc polynomial: crc-15, crc-17, or crc-21
`valid`	`boolean`	`fsm → crc`	pulsed when data should be included in the crc accumulation
`data`	`std_logic`	`fsm → crc`	bit value to feed into the crc engine
`crc`	`crc_vector_t`	`crc → fsm`	current crc register value; fsm reads this when serializing the crc field

1.6.1.2 `can_mac_fsm_tx` `can_mac_fsm_tx` (Figure 6) orchestrates frame transmission, coordinating the serializer (Section 1.6.1.3), bit stuffer (Section 1.6.1.4), and CRC engine (Section 1.6.1.5). On each sample-point strobe from `can_pcs_tx` (Section 1.7.1), the `transmitting_frame` state executes the following sequence:

1. The observed bus polarity is compared against the transmitted bit to detect errors and arbitration loss.
2. In the CAN-FD data phase, any pending SSP observation from the previous bit period is evaluated first [1, Sec. 7.3.4].
3. The next output bit is determined - stuff bit or next frame bit - and the CRC is updated.
4. Any detected error triggers a transition to the appropriate error flag state based on the FCE fault confinement status [1, Secs. 8.1.3–8.1.4].

1.6.1.3 `can_mac_ser_tx` `can_mac_ser_tx` converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM (Figure 7) manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization.

1.6.1.4 `can_mac_bs_tx` `can_mac_bs_tx` performs dynamic bit stuffing for both CAN Classic and CAN-FD frames [1, Sec. 10.6]. It monitors the polarity stream from the FSM (Section 1.6.1.2) and inserts an inverse-polarity stuff bit after every five consecutive identical bits. It also maintains a Gray-coded Stuff Bit Count (SBC) with parity for the CAN-FD CRC field. The data path diagram is shown in Figure 8.

1.6.1.5 `can_mac_crc_tx` `can_mac_crc_tx` provides CRC generation for both CAN Classic and CAN-FD frames. CAN Classic frames use CRC-15, while CAN-FD frames use CRC-17 (data payloads up to 16 bytes) or CRC-21 (data payloads above 16 bytes) [1, Sec. 10.4.2.6]. The FSM selects the appropriate polynomial via the `crc_poly_select` field (Table 13). The data path diagram is shown in Figure 9.

1.6.2 `can_mac_rx`

`can_mac_rx` deserializes the bit stream received from the PCS, performs bit destuffing and CRC verification, and reconstructs the LLC frame for delivery to `can_llc_rx` [1, Sec. 6.6].

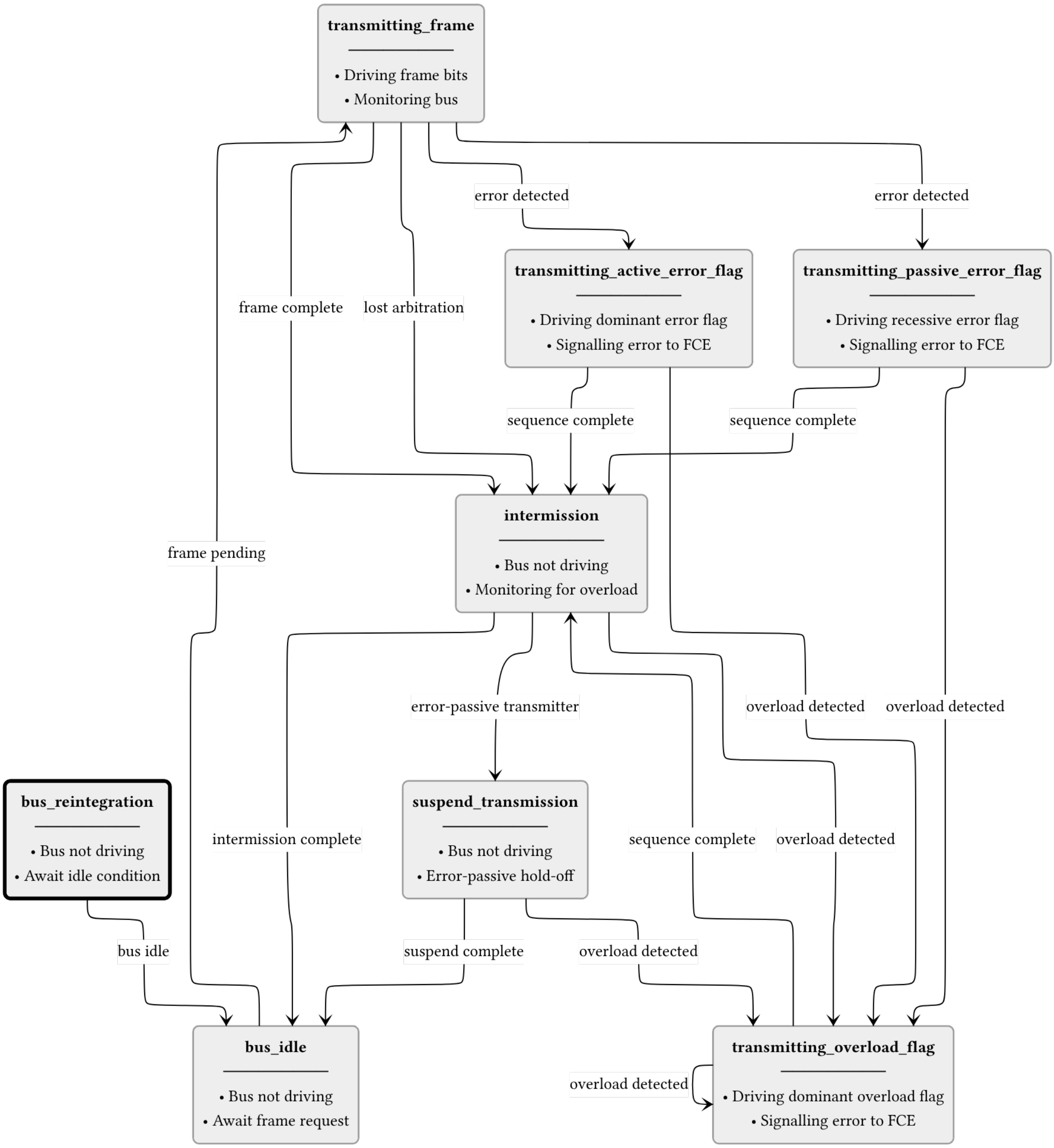

Figure 6: can_mac_fsm_tx FSM. After a successful frame or arbitration loss the FSM passes through intermission before returning to idle. Detected errors branch to either the active (dominant) or passive (recessive) error flag state depending on FCE fault confinement status. Both paths then rejoin the common intermission sequence. Overload frames can be injected from intermission or from either error flag state.

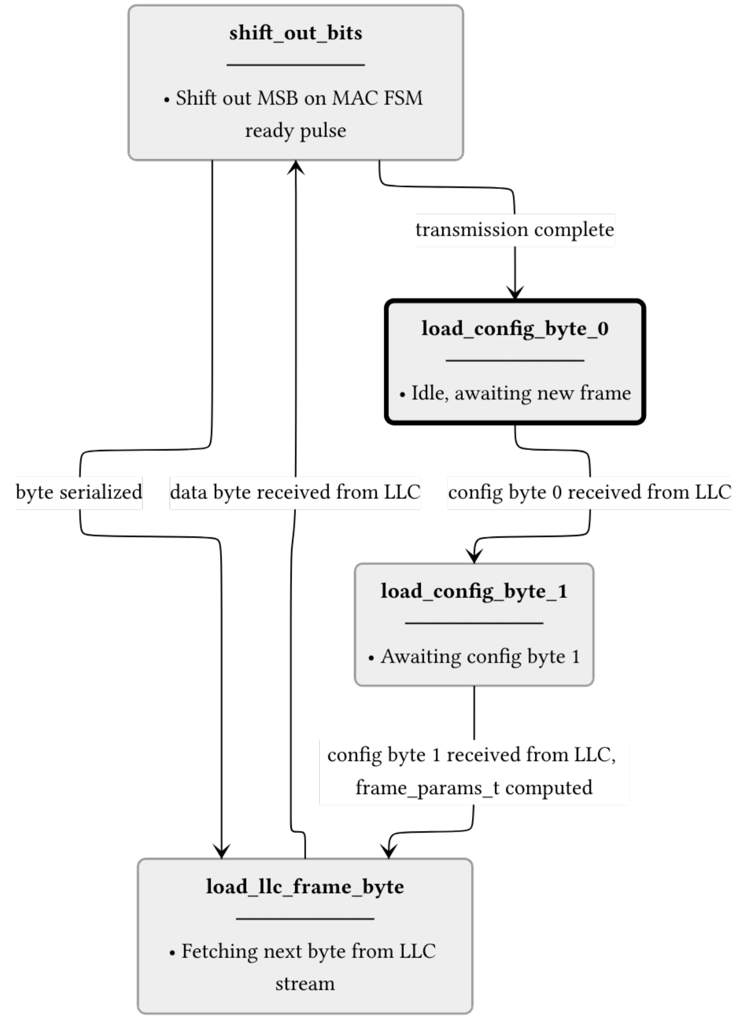

Figure 7: can_mac_ser_tx FSM. The serializer accepts LLC frame bytes over a ready/valid handshake and shifts them out MSB-first to the MAC FSM one bit per ready pulse. Frame parameters are computed once from the two config bytes and cached in frame_params_t for use by the FSM throughout the frame.

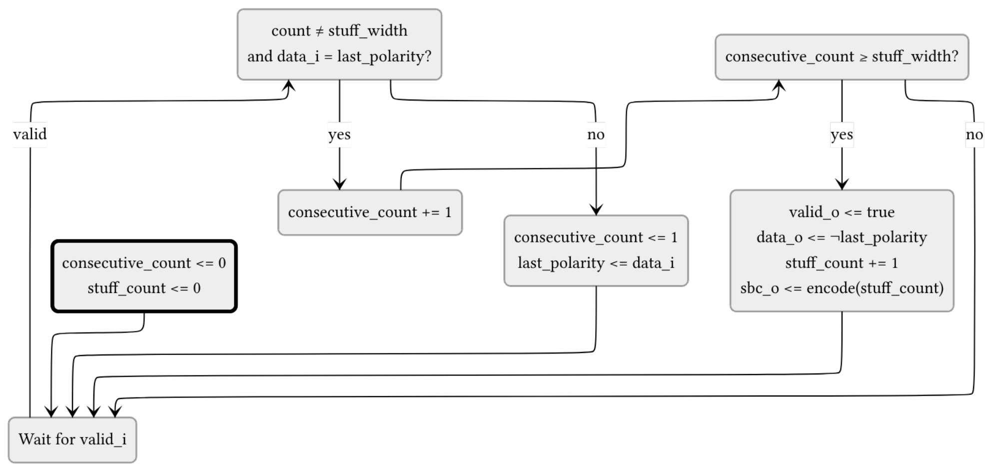

Figure 8: can_mac_bs_tx data path diagram. When valid and data = last_polarity, consecutive_count increments. Otherwise it restarts at 1 with last_polarity updated to data. When consecutive_count reaches stuff_width, valid and inverted data are driven and stuff_count increments. The to_gray() + calc_parity() block encodes stuff_count into the sbc output. The start pulse resets consecutive_count and stuff_count to zero. All outputs are registered.

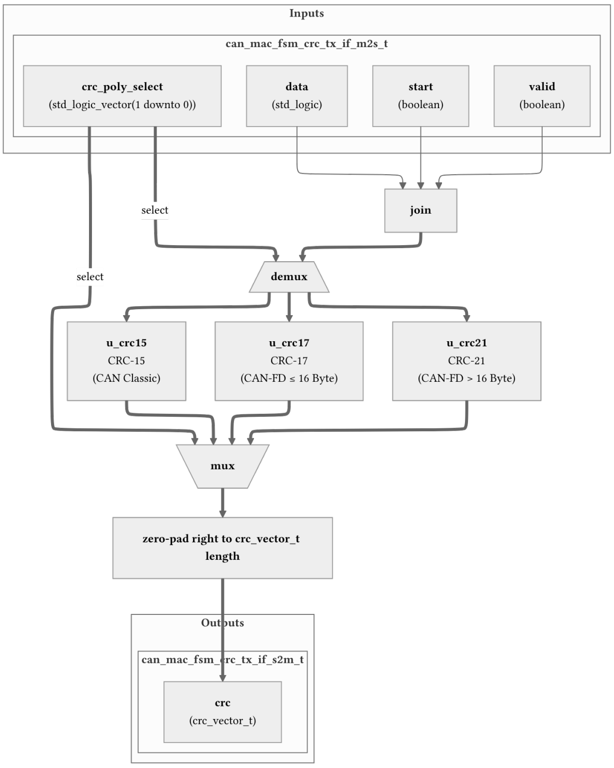

Figure 9: can_mac_crc_tx data path diagram. data, valid, and start are joined and demuxed by crc_poly_select to gate only the selected CRC engine. The mux selects the active engine result, which is zero-padded to the common crc_vector_t width. All outputs are registered.

1.7 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

1.7.1 `can_pcs_tx`

`can_pcs_tx` handles bit timing and Transmitter Delay Compensation (TDC) for the TX path [1, Secs. 7.2–7.3]. It generates the sample point (SP) strobe used by the MAC FSM to advance frame state, and - when TDC is configured - a secondary sample point (SSP) strobe for data-phase bit monitoring. The FSM (Figure 10) has four states reflecting the two bit rates and the TDC measurement window. The `measuring_delay` state determines the Transmitter Delay Compensation Value (TDCV, [1, Sec. 7.3.4]) by observing the TX-to-RX propagation delay, which together with the configured TDCO offset defines the SSP position for subsequent data-phase bits.

1.7.2 `can_pcs_rx`

`can_pcs_rx` implements bit timing and synchronization for the RX path. It performs hard and soft synchronization on the incoming bus signal and generates the sample point strobe used by the MAC RX layer to latch each received bit [1, Secs. 7.2–7.3].

[1] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer*. 2024.

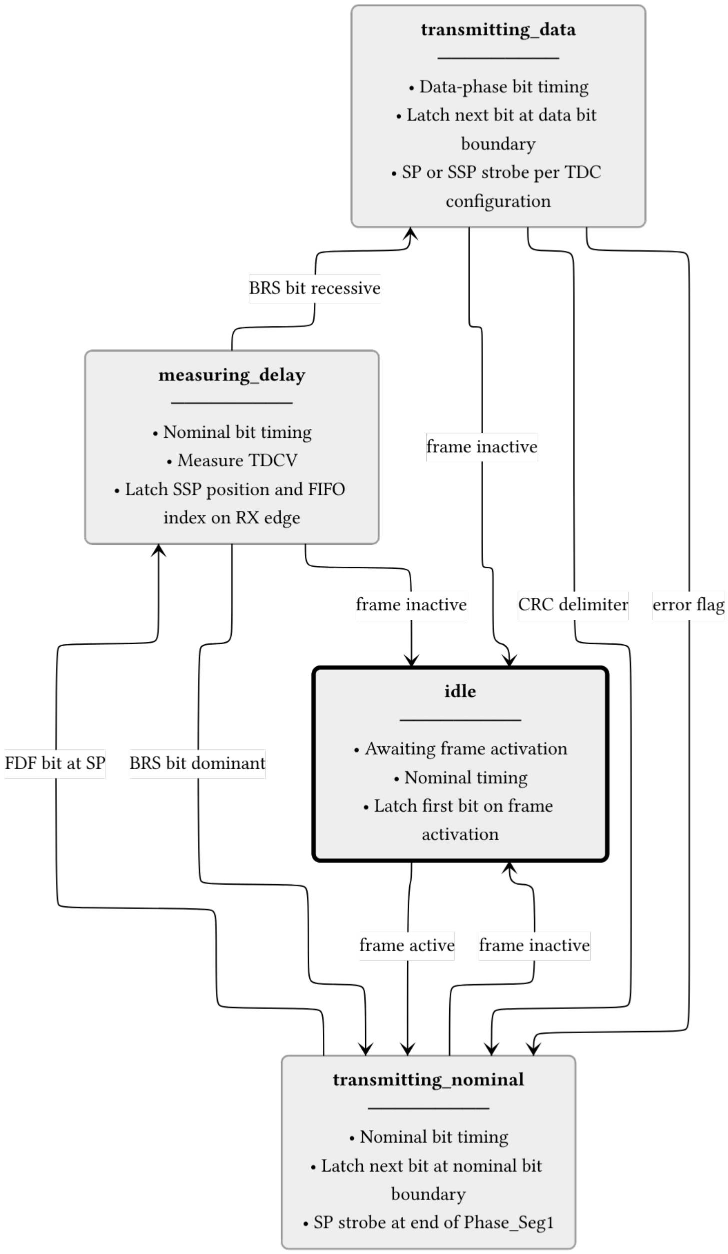

Figure 10: can_pcs_tx FSM. The measuring_delay state is entered on the FDF sample point to measure TDCV (Transmitter Delay Compensation Value, [[1], sec. 7.3.4]). The BRS bit boundary determines whether data-phase timing is used. All non-idle states return to idle when the frame becomes inactive.